GUIDE 40 · GOVERN

Use the threat model to decide how to expose QueryAPIGate

The latest version is at anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?g=40-read-the-threat-model. This copy was made on 4 November 2026.

Time: 20 minutes. **You'll end up with:** a decision about how far to expose QueryAPIGate - your laptop, an internal network, partners on the internet, a browser app, an AI agent - and the settings each step needs, based on what QueryAPIGate does and doesn't defend against.

The full reasoning is in the threat model, and the operational checklist in SECURITY.md. This guide turns them into decisions.

The four things to keep in mind

1. **QueryAPIGate runs SQL against your databases.** Anything that reaches it with the right key can do whatever that key's grants allow. **Every administrator but an auditor can run any SQL on every connection**, like a database superuser; their role only limits what they can change in QueryAPIGate.
2. **The SQL read-only check is a careful pattern-based classifier, not a SQL parser.** It's tested by a fuzz suite and has had one real gap found and fixed. Treat it as a second lock. **The first lock is the database user**: connect with an account that has only the rights the API needs - read-only where it should be.
3. **It doesn't terminate TLS, and isn't a firewall.** A proxy does TLS; your network decides who can reach it.
4. **Its own records aren't tamper-proof.** The audit log and run history are ordinary tables; someone with direct access to the store could change them.

Decide by exposure

On your own machine

- Fine with no API key - but know what that means: **anyone who can reach the port can do anything**. Verified: with no key configured, the server logs at startup - *No API key is configured: every request is allowed, including changing connections and keys* - and the Console's **Alerts** shows a critical *Anyone can use this server*.
- Keep the default bind address (`127.0.0.1`). Don't combine "no key" with `QUERYAPIGATE_CORS_ORIGINS=*`: any website you visit could then query your databases through your browser (guide 38).

On an internal network

- **Set** `QUERYAPIGATE_API_KEY` to a long random value - then give each administrator their own token and role, and keep the shared key as a break-glass key (guide 43).
- **Give everything else a scoped key** - an explicit `connections` list (omitting it means *all*), read-only (guide 13).
- **HTTPS** even internally - keys travel in a header on every request (guide 23).
- **Least-privileged database users**, one per connection.
- Encrypt connection passwords, and back up the store (guide 33).

Partners on the internet

Everything above, and for each partner:

- a key that reaches **only** named saved queries - `connections: []`, no ad-hoc SQL (guide 14);
- a rate limit, an expiry, and `allowed_ips` if their addresses are stable;
- strict parameter rules on those queries (guide 6);
- the proxy restricting `/metrics` and anything else you don't want public;
- `QUERYAPIGATE_TRUST_PROXY` set correctly - wrong in either direction, callers can choose their own apparent address.

Check what a partner can reach with **their** key before handing it over: `GET /catalog` (guide 21).

A browser app

The key in a page is public - anyone can read it from the developer tools. So either a key you'd accept anyone using (named queries only, rate-limited, read-only), or better, signed-in users with `from_claim` parameters so each user only ever sees their own rows. List the app's origin in `QUERYAPIGATE_CORS_ORIGINS` (guide 38).

An AI agent

- A scoped key, not the admin key. Prefer `collections` or `queries` grants - the agent gets saved-query tools only. Add a `connections` grant only if it should run its own SQL, and narrow that with `allowed_tables` (guide 25).
- Writes over MCP only through saved queries an author marked for agents, for a key whose grant allows the write - each approved by a person before it runs, unless it only inserts, and never more rows than the query allows (guide 49). Give agents their own keys: the credential that asked can't approve. `execute_sql` never writes.
- Results are capped (`QUERYAPIGATE_MCP_MAX_ROWS`), every call is in run history with `"transport": "mcp"`.
- Remember that what the agent reads becomes part of its context - and whatever it does next. Give it data you'd be comfortable seeing in its output, and mask what it shouldn't see.
- **A hosted assistant signing users in:** OAuth through your identity provider. Tokens are bound to the MCP server - one issued for anything else is refused - but who gets one, and which role claim it carries, is your provider's configuration: check users can't set their own role claim.
- **What it's told, and what it did:** descriptions steer an agent's SQL, so only owners and admins write them - review imported database comments before relying on them. **Observability → Agents** shows each agent's calls, refusals and frequent statements, and `agent_unusual_volume` warns when one suddenly reads far more than usual.

Limits to know about

These are by design or not yet addressed - plan around them rather than discover them:

- **Owners, admins and developers have no data limits** inside QueryAPIGate - roles limit what they can change, not what they can read. Database grants are the backstop.
- **Names can be probed.** A saved query a key can't reach answers `403` naming its connection; a missing name answers `404`.
- **The MCP server ignores `QUERYAPIGATE_TRUST_PROXY`**, so behind a proxy it can't tell MCP callers' addresses apart.
- **Some connection types have gaps** - no query time limit on generic JDBC, no database-enforced read-only on DuckDB, H2 and JDBC; see the support matrix.
- **A JWT can't be revoked** before it expires - keep token lifetimes short.

- **Rate limits are per process unless Redis is configured** (`QUERYAPIGATE_REDIS_URL`) - fine for protection, not for metering.

When it's the wrong tool

If callers must be able to run *anything*, give them a database account, not QueryAPIGate. If you can't give QueryAPIGate a database user with narrow rights, the read-only check alone shouldn't be what stands between the internet and your data.

Next steps

- The threat model - the reasoning behind all of this.
- SECURITY.md - the checklist, and how to report a vulnerability.
- Deploy with Docker for real.